

Designing a video-based badminton stroke classifier for the V100

A video-based "Architecture 2" for badminton stroke classification should pair **X3D-M** as its **3D CNN backbone** with TrackNetV3 shuttle tracking, fusing both streams via MMTM modules and cross-attention — all trainable end-to-end on a V100 16GB in under 10 hours. This architecture is substantially different from BST's skeleton/keypoint approach by operating directly on RGB player crops, capturing racket angle, grip, and body texture cues invisible to pose estimators. Current state-of-the-art on ShuttleSet (BST-CG-AP) reaches only **~77% accuracy across 35 classes**, ([arXiv](#)) ([ResearchGate](#)) leaving significant room for a complementary video-based system. The VideoBadminton benchmark (2024) already showed SlowFast hitting **82.8% top-1** on 18 stroke classes from RGB alone, ([The Moonlight](#)) confirming the viability of this approach.

Player detection: YOLO26 with BoT-SORT delivers stable crops

YOLO26 (released January 2026) is the recommended detector. Its NMS-free end-to-end design produces deterministic latency and consistent frame-by-frame bounding boxes ([Ultralytics](#)) ([ultralytics](#)) — critical for generating temporally smooth player crops. YOLO26m achieves **~50+ mAP on COCO** while running at real-time speeds, and its ProgLoss + STAL loss functions specifically boost small-object detection for far-court players. The COCO-pretrained person class works out-of-the-box for broadcast badminton since players are prominent subjects. YOLO11m is a proven fallback if YOLO26 integration proves unstable.

([ResearchGate](#))

For tracking, **BoT-SORT** (the Ultralytics default tracker) ([Medium](#)) is ideal because it includes camera motion compensation via frame-to-frame homography estimation ([Miguel-mendez-ai](#)) — essential for broadcast cameras that pan and zoom. Its improved Kalman filter estimates width and height directly (8-tuple state), producing more accurate bounding boxes than ByteTrack. ([Label Ref](#)) On top of BoT-SORT, apply **exponential moving average smoothing** ($\alpha = 0.8$) on bounding box coordinates to further reduce crop jitter.

Practical crop parameters should follow these specifications:

- **Resolution:** Extract crops at **256×256**, then let the training pipeline random-crop to **224×224** (data augmentation) or center-crop at inference. The 224×224 resolution preserves subtle racket/wrist detail that 112×112 would lose, [Nature](#) and X3D-M handles it efficiently.
 - **Frame rate:** Extract at **native broadcast rate** (25 or 30 fps). Let the 3D CNN's temporal stride handle subsampling. Saving all frames preserves flexibility for different sampling strategies.
 - **Which players:** Detect and track **both** players. Crop the **stroke-executing player** for the 3D CNN input. Encode the opponent's court position as a 2D coordinate vector fed to the trajectory stream — BST showed opponent position is predictive of stroke type (e.g., opponent at net → likely lob or clear). Distinguish near/far player by foot y-coordinate relative to court lines. [ResearchGate](#)
 - **Court filtering:** Use court line detection to define the playing area polygon, then discard any person detections outside it (removes spectators, coaches, line judges).
-

Shuttle tracking stays with TrackNetV3, with a trajectory-centric representation

TrackNetV3 remains the gold standard for shuttlecock tracking, achieving **97.5% accuracy** on the Shuttlecock Trajectory Dataset. [ACM Digital Library](#) Its two-module design — trajectory prediction via background estimation with mixup augmentation, plus trajectory rectification via inpainting for occluded frames [Scribd](#) [GitHub](#) — handles the brutal speed of shuttlecocks (up to 426 km/h) [Nycu](#) better than any YOLO-based alternative.

TrackNetV4 (ICASSP 2025) adds a motion-aware fusion mechanism that improves tracking of partially occluded shuttles and is a worthwhile plug-and-play upgrade, [Tracknetv4](#) but TrackNetV3 alone is sufficient.

For downstream representation, use **per-frame normalized 2D coordinates** (x/width , y/height) — this is exactly what BST uses [ResearchGate](#) and it achieves SOTA results without requiring court homography estimation. Track at full native frame rate; at 30 fps the shuttle moves ~4 meters between frames during smashes, so every frame matters. Optionally compute **velocity vectors** as finite differences ($v_t = \text{pos}_t - \text{pos}_{t-1}$) as auxiliary features, since shuttle speed at the hit frame strongly discriminates stroke types (smash vs. drop shot). For frames where the shuttle is occluded, TrackNetV3's built-in InpaintNet rectification module fills gaps automatically. [ACM Digital Library](#) [GitHub](#) Store a per-frame confidence flag so downstream models can weight uncertain positions lower.

The trajectory window should be **wider than the video window** — include shuttle positions spanning 2–3 strokes before and after the target stroke. BST demonstrated that incoming trajectory (speed, angle from the opponent's previous stroke) is "critical information" for classifying the response stroke type. [arXiv +2](#)

X3D-M is the optimal backbone for this constraint envelope

After evaluating seven model families against the specific requirements — fine-grained discrimination on ~**25k samples**, V100 16GB memory, and 224×224 player crops — **X3D-M emerges as the clear winner**, with R(2+1)D-18 as runner-up and VideoMAE ViT-S as a high-accuracy alternative.

X3D-M (Meta) has just **3.8M parameters** and **6.2 GFLOPs** per clip. [Read the docs](#) This tiny parameter count gives an exceptionally favorable sample-to-parameter ratio of ~6.5:1 on 25k samples, virtually eliminating overfitting risk. It natively accepts inputs from 160×160 to 224×224 with 16 frames, fits **batch sizes of 16–32 on V100 16GB with FP16** (no gradient checkpointing needed), and trains in approximately **5–10 hours for 30 epochs** on 25k samples. Kinetics-400 pretrained weights are available through PyTorch Video and PySlowFast. [GitHub](#) [PyTorch](#) Its channel-wise separable 3D convolutions [TheCVF](#) with squeeze-and-excitation blocks provide good spatial feature extraction; [TheCVF](#) the tradeoff is somewhat weaker temporal modeling than transformers, which we compensate with downstream temporal attention.

Model	Params	GFLOPs	V100 batch (FP16)	Training time	SSv2	Practical score
X3D-M	3.8M	6.2	16–32	5–10 h	~moderate	★★★★★
R(2+1)D-18	31.5M	40.7	8–16	15–30 h	moderate	★★★★
VideoMAE ViT-S	22M	~70	4–8 (+ checkpointing)	30–100 h	~70%	★★★★
SlowFast R50 4×16	34M	36	4–8	15–30 h	~60%	★★★★
Video Swin-T	28M	88	2–4 (+ checkpointing)	30+ h	~65%	★★
UniFormer-S	21.7M	42	4–8	20–40 h	~67%	★★★★
TimeSformer	121M	590+	1–2	impractical	62.5%	★

R(2+1)D-18 is the simplest to integrate (two lines via torchvision's `r2plus1d_18(weights=Kinetics400_V1)`) and its factored spatial-then-temporal convolutions provide strong temporal discrimination. However, its **31.5M parameters** require aggressive regularization and partial freezing on 25k samples. **VideoMAE ViT-S** offers superior fine-grained performance (the VideoMAE family achieves remarkable results on datasets as small as 3.5k videos) [GitHub](#) but requires 224×224 input, gradient checkpointing, and substantially longer training. It becomes attractive for A100 hyperparameter sessions.

Use Kinetics-400 pretrained weights for all models — the domain gap from Kinetics to badminton strokes is moderate (both involve human motion), and pretrained features dramatically accelerate convergence.

Asymmetric temporal windows centered on racket contact

The temporal windowing strategy significantly impacts classification quality. The literature and BST's own ablations point toward a **fixed asymmetric window** as the primary approach, with the option to provide wider trajectory context separately.

Recommended video window: -24 to +8 frames around the hit frame at 30 fps (~0.8s pre-contact + 0.27s post-contact = 32 frames total). The pre-contact phase — backswing, body positioning, footwork — is the **primary discriminator** between stroke types. A drop shot and a smash originate from similar court positions but have radically different preparation. Post-contact follow-through provides confirmation but carries less discriminative information. This 32-frame clip maps directly to X3D-M's 16-frame input via stride-2 sampling, always anchoring the contact frame.

BST's variable-length window (previous stroke hit → next stroke hit) captures rally context but creates variable sequence lengths that complicate batching and 3D CNN processing.

[arXiv](#) It works well for transformers processing token sequences but less naturally for fixed-input 3D CNNs. The BST authors themselves found that their variable segmentation caused TemPose's performance to *drop* due to architectural constraints. [arXiv](#)

For frame sampling within the window, use **dense sampling with contact-frame anchoring** — ensure the hit frame is always included and not randomly dropped during temporal augmentation. Sparse TSN-style sampling risks missing the critical contact moment in fast badminton strokes. [TheCVF](#) If using X3D-M's 16-frame input from a 32-frame window, sample every 2nd frame with the contact frame guaranteed at position 12 (of 16).

For the **trajectory stream**, use a wider window spanning **2-3 strokes** before and after the target, providing rally context that the tight video window cannot capture. This asymmetry between modality windows is a key design insight: the video captures fine-grained stroke mechanics while the trajectory captures tactical context.

Fusing video and trajectory through MMTM plus cross-attention

The fundamental challenge is fusing **dense video features** (spatial-temporal tensor from a 3D CNN) with **sparse trajectory data** (a sequence of 2D coordinates). After evaluating seven fusion strategies against the literature, a **two-stage fusion approach** emerges as optimal.

Stage 1 — MMTM (Multimodal Transfer Module) at intermediate layers: Insert 2–3 MMTM modules at the last residual stages of X3D-M. MMTM uses squeeze-and-excitation to generate cross-modal recalibration signals, allowing trajectory information to modulate which visual features the CNN emphasizes. (arXiv) Critically, MMTM was designed specifically for "modalities with different spatial dimensions" — the squeeze operation collapses spatial dimensions, making it natural to fuse a 3D feature volume with a 1D trajectory vector. (IEEE Xplore) It adds negligible computation (two FC layers per module) and can be inserted into pretrained architectures with minimal changes. (arXiv)

Stage 2 — Cross-attention after feature extraction: After the 3D CNN produces temporal feature vectors and a small TCN encodes the trajectory sequence, apply **one layer of cross-attention** where video features query trajectory features and vice versa. This mirrors BST's CrossTrans module, which was shown to be a critical component — it "calculates the similarity between pose and trajectory, using similarity values to pick value arrays from trajectory." (arXiv) The cross-attention naturally handles temporal alignment between the two streams.

Add gated fusion for robustness: A learned sigmoid gate on the trajectory branch allows the model to suppress trajectory influence when shuttle tracking fails (occlusion, detection gaps). (ScienceDirect) This is important because TrackNetV3, while excellent, is not infallible during fast rallies. (ACM Digital Library)

Early fusion (concatenating trajectory as extra CNN channels) is **not recommended** — the spatial density mismatch between video pixels and coordinate pairs is too severe for convolutional kernels. Pure late fusion (concatenating final feature vectors) works as a baseline but cannot capture how trajectory context should influence *which* visual features matter. (arXiv) Bottleneck fusion (MBT) is excellent but assumes transformer-based encoders for both streams. (OpenReview)

For temporal modeling after the 3D CNN, **replace the standard temporal global average pooling (TGAP) with temporal attention pooling** — a single self-attention layer over temporal positions. Kalfaoglu et al. (ECCV 2020) showed this consistently improves all tested 3D CNN architectures by allowing the model to identify the most discriminative frames rather than treating all equally. (arXiv) No additional TCN or transformer stack is needed on the video side for single-stroke classification, since the 3D CNN's receptive field already covers the full 1-second clip. The TCN is reserved for encoding the trajectory stream, consistent with BST's validated architecture. (arXiv)

Training strategy: progressive unfreezing on V100 16GB

The literature strongly indicates that **pure frozen backbones are catastrophic for fine-grained temporal tasks** — AIM (ICLR 2023) showed frozen ViT on Something-Something v2 achieves only 15.1% versus 59.5% with full fine-tuning. (arXiv) Kinetics-pretrained features capture "what sport" but not "what specific stroke." Conversely, X3D-M's 3.8M parameters are small enough that full fine-tuning is safe on 25k samples.

Recommended three-stage training protocol:

1. **Stage 1 — Head warmup (5 epochs):** Freeze X3D-M backbone entirely. Train only the classification head, fusion modules (MMTM, cross-attention), and trajectory TCN. Learning rate **1e-3** with Adam. This establishes a good classifier and fusion mechanism before disturbing pretrained features. Memory usage: ~50% less than full fine-tuning, batch size 32+.
2. **Stage 2 — Full fine-tuning (25 epochs):** Unfreeze all X3D-M layers. Learning rate **1e-4** with cosine annealing to 1e-6. Apply strong regularization: random spatial crop (224 from 256), horizontal flip, temporal jitter (± 2 frames), color jitter, **Mixup ($\alpha=0.2$)**, **CutMix**, weight decay 5e-4, dropout 0.5 on the head. Keep all BatchNorm layers frozen (BN statistics from Kinetics pretraining are more stable than what 25k samples can estimate). (ESIEE) Batch size 16 with FP16 mixed precision.
3. **Stage 3 — A100 refinement (optional):** Use A100 40GB sessions for hyperparameter search over learning rate, temporal window parameters, fusion module placement, and Mixup alpha. Batch size 32-64 enables faster iteration. Budget ~12 hours per session for grid search over 3-4 key hyperparameters.

Mixed precision is non-negotiable on V100: The V100's Tensor Cores deliver ~**125 TFLOPS FP16** versus ~15 TFLOPS FP32 — an 8× throughput gain for matrix operations. Use PyTorch's `torch.cuda.amp.autocast()` and `GradScaler`. Gradient checkpointing is unnecessary for X3D-M but essential if experimenting with R(2+1)D-18 or VideoMAE on V100 (saves ~60% activation memory at ~25% speed cost). Gradient accumulation over 2 steps (effective batch 32 from micro-batch 16) is recommended for training stability.

For data loading, **decode videos to individual frames offline** and store as JPEG/PNG or in LMDB format. Decoding video on-the-fly is a common bottleneck that can waste 30-50% of training time. Use 4-8 dataloader workers.

Recent literature validates this design and reveals remaining gaps

The most relevant recent work confirms the viability and fills in key design details:

- **VideoBadminton (IEEE BigData 2024)** directly benchmarked 3D CNNs on badminton RGB video: (arXiv) SlowFast achieved **82.8% top-1** on 18 classes from 7,822 clips, outperforming Video Swin, TimeSformer, MViT-V2, and skeleton-based models (ST-GCN, PoseC3D). (arXiv) (The Moonlight) This validates that RGB-based approaches can match or exceed skeleton methods.
- **FineBadminton (ACM MM 2025)** introduced multi-level semantic annotations (foundational actions → tactical semantics → decision evaluation) (Hugging Face) (arXiv) and a hit-centric keyframe selection strategy (ResearchGate) (arXiv) that aligns with our asymmetric windowing approach.
- **V-JEPA 2 (2025)** is a self-supervised video model that excels specifically at fine-grained motion understanding on Something-Something v2 (arXiv) — it could serve as a powerful pretrained backbone if available in a small variant, though current model sizes may exceed V100 constraints.
- **FTP framework (2024)** achieved **83.4% on SSv2** by aligning video encoders with vision-language model outputs through four-tiered prompts (action category, components, description, context). (arXiv) This suggests augmenting stroke classification with textual stroke descriptions during training could boost fine-grained discrimination.
- **No paper has significantly outperformed BST on ShuttleSet** as of early 2026. BST-CG-AP at ~77% accuracy across 35 classes remains SOTA, (ResearchGate) suggesting substantial headroom for improvement — particularly from an RGB-based approach that captures visual details (racket face angle, string contact point, grip position) invisible to 2D skeleton extractors. (arXiv)

The multi-label decomposition approach from table tennis research (Sensors 2025) is worth considering: instead of classifying 35 monolithic stroke types, decompose into sub-tasks (overhand/underhand × power level × direction) and use multiple output heads. (MDPI) This reduces per-head class count and can improve accuracy on confusable categories like smash vs. wrist smash. (arXiv)

Conclusion

The complete architecture chains together proven, efficient components: **YOLO26m** + **BoT-SORT** for stable player detection and tracking → **256×256 crops resized to 224×224** → **X3D-M** (3.8M params, Kinetics-pretrained) processing 16 frames from an asymmetric window (−24 to +8 frames around contact) → **MMTM fusion** at intermediate layers (arXiv) with shuttle trajectory encoded by a lightweight TCN → **cross-attention** between video and trajectory features → **temporal attention pooling** replacing TGAP → classification head. TrackNetV3 provides shuttle coordinates (Scribd) (GitHub) on a wider trajectory window spanning 2–3 surrounding strokes. The entire system trains in under 10 hours on a single V100 16GB with FP16 mixed precision, progressive unfreezing, and standard augmentation.

Three non-obvious insights from this research deserve emphasis. First, X3D-M's extreme parameter efficiency (3.8M) is not a compromise but an *advantage* on 25k samples — larger models like SlowFast and Video Swin risk overfitting and train 3–10× slower with no guaranteed accuracy gain. ([Read the docs](#)) Second, the trajectory stream's temporal window should be deliberately wider than the video window, creating an information asymmetry where fine-grained visual mechanics are complemented by broad tactical context. Third, frozen BatchNorm layers during fine-tuning is a small detail with outsized impact — Kinetics-estimated batch statistics are far more reliable than what 25k badminton samples can provide, and unfreezing BN is a common source of training instability in video fine-tuning.